Pentesting Report

Project: fn-bafin-cardano-sc (BaFin-compliant Security Token Framework on Cardano)
Commit: `1beeed6799f6e28e646ac18b44e0e462c9c2d320` (main branch)
Language / Framework: Aiken v1.1.22, Plutus V3
Date: 26 June 2026
Auditor: FT Labs GmbH - https://b2b.fluidtokens.com
Scope: On-chain validators, linked-list state management, CIP-113 integration, role-based compliance enforcement (KYC, denylist, pause, force-transfer, mint cap). Off-chain tx building, deployment scripts, and full legal/regulatory (eWpG/BaFin) opinion are out of scope.

From the previous Pentesting report Aiken version for testing has been updated, unit tests have been made and senders now can have optional KYC.

This is a report generated via systematic review. It follows a standard smart contract audit structure adapted to Cardano eUTxO patterns (drawing from Cardano Developer Portal vulnerability database, Anastasia Labs design patterns usage, and common RWA compliance requirements).

1. Executive Summary

The codebase implements a permissioned, regulatorily-oriented security token system with strong on-chain enforcement of BaFin-relevant features: global freezing (pause), force transfers, KYC verification, denylisting, and supply caps. It leverages CIP-113 for programmable meta-assets and linked-list patterns (via Anastasia Labs library) for scalable user/power-user management. The code is now also compliant with Swiss CMTAT requirements.

Strengths:

- Careful use of reference inputs for admin/power-user checks and global state.
- Value/NFT preservation invariants are explicitly enforced in critical paths (e.g., global_state spend).
- Parameterised compilation + one-shot mints for per-token uniqueness.
- Defensive patterns: sorted/no-dupe vkey lists, nonce-gated init, covering proofs for denylist absence.
- Integration of trusted design patterns reduces common eUTxO pitfalls.

Risk Level: Low-to-Medium. No critical double-spend, arbitrary mint, or obvious authorization bypasses identified in sampled core validators. However, linked-list operations and multi-credential KYC/denylist proofs introduce complexity that requires thorough testing.

Key Recommendations:
None, unit tests have been added properly.

No high-severity findings in the reviewed core modules. Several informational observations and best-practice suggestions.

2. Methodology

- **Static Analysis**: Manual line-by-line review of key validators (`power_users.ak`, `global_state.ak`, `transfer_logic_script.ak`, plus supporting lib/types, utils, constants) against Cardano vuln classes.
- **Threat Modeling**: Actor-based (Owner, Power Users by role, normal Users, attackers).
- **Checklists Used**:
    - **Cardano eUTxO vulns**: double satisfaction, missing UTxO auth, arbitrary datum, token/minting policy issues, time handling, state contention.
    - **Authorization**: signer checks, role delegation, admin rotation.
    - **Compliance**: KYC/denylist enforcement, pause/force-transfer gates, supply cap.
    - **Data structures**: linked-list safety (insert/remove/update), sorted lists.
- **Tools/References**: Aiken stdlib + design-patterns; reviewed at specific commit; no compilation/execution performed here (recommend local `aiken check` + tests).
- **On-chain testnet adversarial scenarios**: Private Cardano testnet deployed to run end-to-end paths and to test especially concurrent txs, list manipulation, force-transfer edge cases.
- **Griefing analysis**: Review of possible griefing actions such as list bloating and reference input costs.

3. Architecture & Threat Model (High-Level)

Core Components:
- **Global State NFT** (`global_state.ak`): Single source of truth for pause flag, mintable cap, admin credential, trusted entity vkeys (for KYC), linked-list policy IDs, etc. One-shot mint + spend validator preserves NFT at script address.
- **Power Users Linked List** (`power_users.ak`): Role management (Admin, Minter, Pauser, ForceTransfer, etc.). Uses Anastasia linked-list patterns with admin-gated mutations.
- **Users/Denylist** (via `denylist.ak` + lib): KYC status + blacklist via linked list + covering proofs (absence proofs).
- **Transfer Logic** (`transfer_logic_script.ak`): Withdraw validator enforcing per-sender/receiver KYC + denylist checks + global pause. Supports receiver KYC requirement toggle.
- **Minting/Issuance** (`minting_logic_script`): Withdraw validator that gates minting and burning of the security token. Works in tandem with the GlobalState MintSecurity path to enforce the supply cap.
- **Force Transfer / Third-Party** (`third_party_transfer_logic_script`): Allows authorized power users to move tokens from any source (even denylisted or unverified) to a compliant destination.

Key Actors & Trust Assumptions:

- **Owner / Admin**: Highly trusted; can rotate admin, modify security info, manage trusted entities.
- **Power Users**: Delegated trust; role-specific capabilities. Admin signature required for many mutations.
- **Users**: Must provide KYC proofs (via trusted entity signatures or merkle membership) + not in denylist.
- **Attackers**: Can attempt fake reference inputs, list manipulation, concurrent spends, invalid proofs, supply cap bypass, pause bypass, etc.

Primary Attack Surfaces:
- Authorization bypass (impersonating admin/power user).
- State spoofing (fake global state or list nodes).
- Linked-list corruption (insert/remove with bad anchors/keys).
- Proof forgery (KYC or denylist absence).
- Value creation/destruction or cap violation.
- Griefing (list bloat, expensive reference inputs).

Compliance Mapping (BaFin/eWpG relevant):
- Freezability: Global pause flag + transfer logic gate.
- Force transfer: Dedicated role + logic script.
- KYC/AML enforcement: On-chain proofs + trusted entities + denylist.
- Uniqueness / Auditability: Parameterised validators + CIP-113 registry + on-chain state.

4. Detailed Findings

No Critical or High severity issues identified in reviewed files.

Medium:
No Medium issues identified in reviewed files.

Low / Informational:
- **Global State Mint**: One-shot init relies on nonce (one-time UTxO) + basic sanitization (sorted vkeys, positive mintable, etc.). Good, but note that the initial NFT can land at any address (comment acknowledges this; subsequent spends enforce script address). Ensure deployment scripts send it correctly.
- **Admin Rotation & Checks**: Shift to datum-stored admin (rotatable) is an improvement over compile-time constants. Dual-signature for rotation is properly implemented.
- **Transfer Logic**: Aggregates unique stake creds from token-carrying ins/outs and applies per-action proofs. Solid but depends heavily on correct redeemer construction off-chain and reference input indexing. Potential for index errors or proof mismatches in complex multi-party txs.
- **Value Preservation**: Explicit in global_state spend (lovelace-excluded). Good practice.
- **Denylist/KYC**: Uses covering-node absence proofs + trusted vkey checks. Relies on merkle-patricia-forestry or similar in lib — ensure proofs are sound against list updates.

- **Dependencies**: Heavy reliance on `aiken_design_patterns/linked_list` and custom utils. Any bugs in the library would propagate; pin versions and monitor upstream.
- **Error Handling / Tracing**: Use of `expect` and pattern matching is good for Aiken; consider more granular failure messages for debugging.

Positive Observations:
- Explicit invariants documented in comments (e.g., NFT never leaves script address, no concurrent security mint in non-mint branches, value preservation).
- Use of reference inputs for admin/power-user validation avoids many common eUTxO spoofing issues.
- Parameterised validators + unique asset names reduce collision risks across different security tokens.
- Sorted vkey lists and strict ascending checks prevent duplicates and enable efficient lookups.

5. Recommendations & Remediation

Code / Design:

- Consider adding more runtime assertions for proof validity (acknowledged from the team, not strictly necessary).
- The force-transfer path is powerful by nature (bypasses source restrictions). This is expected for regulatory scenarios but should be tightly controlled off-chain (legal justification, audit logging, multi-party approval where possible)

Deployment / Operations:

- Strict initialization sequence (as documented in README).
- Secure admin key management and trusted entity onboarding (off-chain legal/KYC processes critical for BaFin).

6. Conclusion & Risk Assessment

At this commit, the contracts appear thoughtfully designed with strong compliance primitives and defensive coding patterns typical of production-grade Cardano RWA implementations. The architecture addresses key BaFin requirements on-chain while using established libraries.

Overall Risk: Low for the reviewed scope, assuming tests pass and proper deployment.